Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 07

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Identificar la funcionalidad avanzada del ORM de Django (transacciones, expresiones F, agregaciones y QuerySets personalizados) dentro de una aplicación empresarial.**
2. **Registrar operaciones de forma segura con transaction.atomic() y F(), y construir reportes con aggregate() y annotate() sobre modelos relacionados.**
3. **Encapsular consultas de negocio en un QuerySet personalizado y optimizar su rendimiento con select_related y prefetch_related, midiendo el número de consultas.**

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la Semana 7 (ORM avanzado: transacciones y F(), aggregate() y annotate(), QuerySet personalizado y problema N+1) antes del desarrollo del laboratorio.

III. Normas empleadas

- No aplica

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.
- La aplicación desarrollada desde la Semana 3 (CRUD con URLs, Views y Templates), ampliada con relaciones en la Semana 4, registrada en el Django Admin en la Semana 5 y con Templates que heredan de base.html desde la Semana 6, y, cuando corresponda, la investigación propia de siete entidades.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código (models.py, views.py y Template correspondiente) y del resultado en el navegador o en el shell de Django, incluyendo el número de consultas medido antes y después de optimizar.
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

VII. Procedimiento

Desde la Semana 3, cada estudiante ya cuenta con URLs, Views y Templates propios que implementan CRUD completo, ampliados en la Semana 4 con relaciones, registrados en el Django Admin en la Semana 5 y con Templates que heredan de base.html desde la Semana 6. En esta sesión no se agregan entidades nuevas: se aprovechan las que ya existen para registrar operaciones de forma segura y generar reportes con consultas avanzadas del ORM. A diferencia

de la Semana 6, sí se crean una View y un Template nuevos para mostrar el reporte; el Template debe heredar de base.html.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante retoma la aplicación de las Semanas 4 a 6 y aplica transacciones, agregaciones, un QuerySet personalizado y la optimización de consultas. En la segunda parte, deberá hacer lo mismo sobre su investigación propia (las siete entidades registradas en el Admin en la Semana 5), cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda que las consultas avanzadas no reemplazan al ORM que ya conoce: lo aprovechan. Las mismas relaciones de la Semana 4 permiten calcular totales, agrupar resultados, encapsular reglas de negocio y reducir el número de consultas a la base de datos. En clase la teoría se demuestra con una aplicación de cafetería; cada estudiante la traslada a su propia aplicación con ayuda de la siguiente tabla de equivalencias.

Concepto trabajado en clase	**Equivalente en su aplicación**
Campo entero que se descuenta (en clase: stock de Producto)	Un campo numérico entero de su aplicación (existencias, cupos, unidades disponibles, copias).
Campo de estado (en clase: estado de Pedido)	Un campo de texto con valores como PENDIENTE, PAGADO, ACTIVO o DEVUELTO.
Atributos del modelo intermedio (en clase: cantidad y precio_unitario de DetallePedido)	Los atributos propios de su modelo intermedio N:M (cantidad, monto, multa, horas, fecha).
Operación que modifica varios modelos (en clase: registrar un pedido)	Una operación de su aplicación que crea o modifica más de un registro a la vez.

Cliente (navegador)

→ URL nueva (ej. /reporte/) en urls.py

→ View nueva (views.py)

→ QuerySet personalizado (Manager)

→ Django ORM (aggregate / annotate)

→ SQLite

→ Template nuevo (extends base.html)

→ Response

Al finalizar el laboratorio, el estudiante deberá ser capaz de registrar una operación con transaction.atomic() y F(), construir reportes con aggregate() y annotate(), encapsular consultas en un QuerySet personalizado con as_manager(), y medir y reducir el número de consultas con select_related y prefetch_related.

Enunciados

PARTE 1 — Consultas avanzadas sobre la aplicación de las Semanas 4 a 6

Ejercicio 1 — Recuperar la aplicación y ubicar sus campos clave

Abra la aplicación desarrollada hasta la Semana 6 e identifique su entidad principal y sus tres relaciones (OneToOneField, ForeignKey y ManyToManyField con through). Con ayuda de la tabla de equivalencias, ubique tres campos: un campo entero que pueda descontarse, un campo de estado y un atributo numérico del modelo intermedio. Si alguno no existe, agréguelo al Model,

ejecute makemigrations y migrate, y documente el cambio con una captura del archivo de migración generado.

Respuesta:

La entidad principal es la cita. Sus tres relaciones son la ficha clínica (uno a uno), el veterinario (uno a muchos, la cita guarda la clave foránea) y los insumos (muchos a muchos a través del modelo intermedio consumo de insumo).

Los tres campos que pide el laboratorio ya existían desde las semanas anteriores: el stock del insumo es el campo entero que se descuenta, el estado de la cita (Pendiente, Confirmada o Atendida) es el campo de estado, y la cantidad consumida junto con el precio unitario son los atributos numéricos del modelo intermedio. Por eso no hizo falta una migración nueva: se comprobó que no hay cambios pendientes y que las dos migraciones de la app ya están aplicadas.

vet/models.py — Cita: campo de estado, ForeignKey y ManyToManyField con through

```
    mascota = models.CharField(max_length=100)
    dueno = models.CharField(max_length=100)
    servicio = models.CharField(max_length=20, choices=SERVICIOS)
    fecha = models.DateField()
    hora = models.TimeField()
    estado = models.CharField(max_length=20, choices=ESTADOS, default='Pendiente')
    objects = CitaQuerySet.as_manager()

    # --- Ejercicio 3: UNO A MUCHOS -------------------------------------
    # La FK va aqui porque Cita es el lado "muchos": un veterinario atiende
    # muchas citas, pero cada cita la atiende un solo veterinario.
    # SET_NULL: si un veterinario deja la clinica, su historial de citas NO
    # debe borrarse; la cita queda registrada como "sin asignar".
    # null=True es ademas obligatorio aqui: la tabla ya tenia filas cuando se
    # agrego la columna, y sin null Django pediria un valor por defecto.
    veterinario = models.ForeignKey(
        Veterinario,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='citas',
    )

    # --- Ejercicio 4: MUCHOS A MUCHOS con modelo intermedio ------------
    # La relacion guarda datos propios (cantidad y precio), por eso NO puede
    # ser un M2M simple: necesita el modelo intermedio ConsumoInsumo.
    insumos = models.ManyToManyField(
        Insumo,
        through='ConsumoInsumo',
        related_name='citas',
        blank=True,
    )
```

vet/models.py — Insumo: campo entero que se descuenta

```
class Insumo(models.Model):
    """Ejercicio 4 - el otro extremo de la relacion muchos a muchos."""
    nombre = models.CharField(max_length=80, unique=True)
    unidad = models.CharField('Unidad de medida', max_length=20, default='unidad')
    stock = models.PositiveIntegerField(default=0)

    def __str__(self):
        return self.nombre
```

vet/models.py — OneToOneField y atributos del modelo intermedio

```python
class FichaClinica(models.Model):
    """Ejercicio 2 - UNO A UNO: ficha complementaria de la cita.

    Solo existe si la cita fue atendida y, como maximo, una por cita.
    CASCADE: una ficha clinica sin su cita es un dato huerfano; se elimina
    junto con ella.
    """
    cita = models.OneToOneField(
        Cita,
        on_delete=models.CASCADE,
        related_name='ficha',
    )
    peso_kg = models.DecimalField('Peso (kg)', max_digits=5, decimal_places=2)
    diagnostico = models.TextField()
    tratamiento = models.TextField(blank=True)
    proximo_control = models.DateField(null=True, blank=True)

    class Meta:
        verbose_name = 'Ficha clínica'
        verbose_name_plural = 'Fichas clínicas'

    def __str__(self):
        return f'Ficha de {self.cita.mascota}'


class ConsumoInsumo(models.Model):
    """Ejercicio 4 - modelo intermedio (through) de Cita <-> Insumo.

    Guarda dos atributos que NO pertenecen ni a Cita ni a Insumo: cuantas
    unidades se usaron y a que precio se cobraron ese dia.
    """
    cita = models.ForeignKey(Cita, on_delete=models.CASCADE,
                             related_name='consumos')
    insumo = models.ForeignKey(Insumo, on_delete=models.PROTECT,
                               related_name='consumos')
    cantidad = models.PositiveIntegerField(default=1)
    precio_unitario = models.DecimalField(max_digits=7, decimal_places=2)
```

Terminal — no hay migraciones pendientes

```
PS ...\django_project\src> python manage.py makemigrations --check --dry-run
No changes detected
PS ...\django_project\src> python manage.py showmigrations vet
vet
 [X] 0001_initial
 [X] 0002_insumo_veterinario_fichaclinica_consumoinsumo_and_more
```

Ejercicio 2 — Cargar datos de prueba

Desde el Django Admin (Semana 5), registre datos suficientes para que los reportes tengan resultados: al menos cinco registros de la entidad principal, tres de la entidad de la relación 1:N y

ocho del modelo intermedio N:M, con estados y cantidades variadas. Documente con una captura el listado en el Admin.

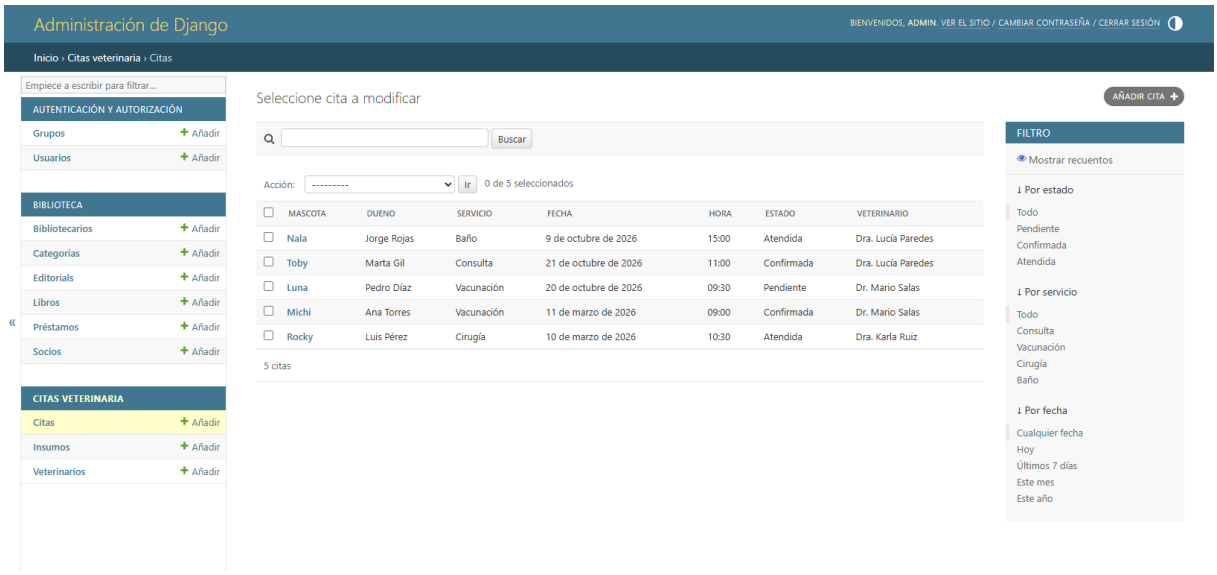
Respuesta:

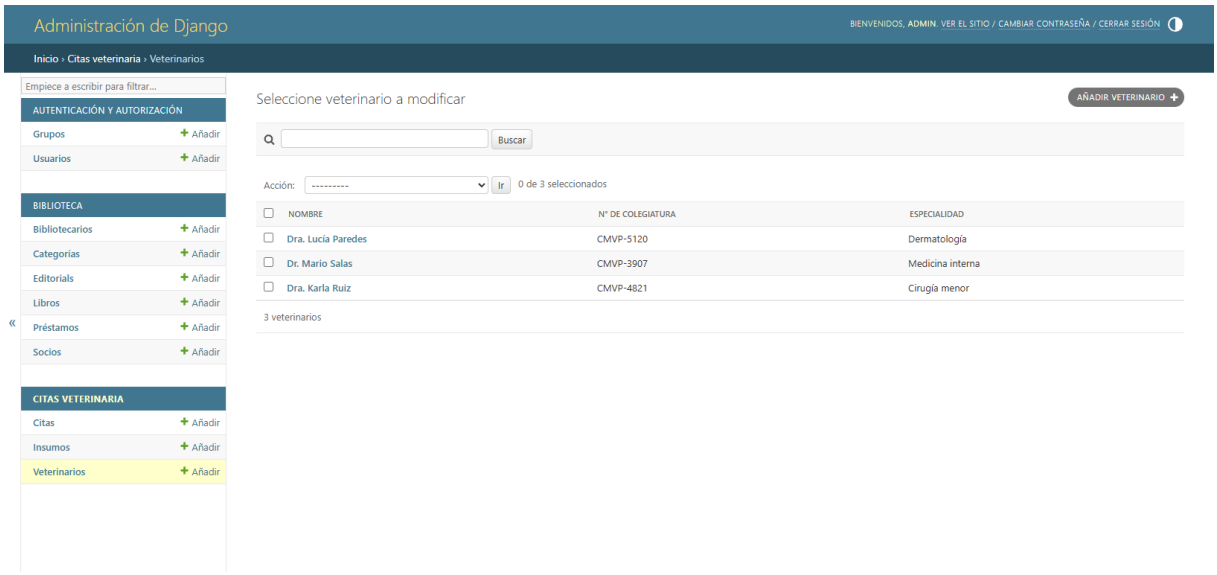
Se dejó un script de carga que se puede ejecutar varias veces sin duplicar datos. La base quedó con 5 citas con estados variados, 3 veterinarios, 4 insumos y 8 consumos de insumo, que cumple el mínimo de la guía (5, 3 y 8). Las citas de Luna y Toby tienen fecha de fines de octubre para que el reporte tenga citas próximas.

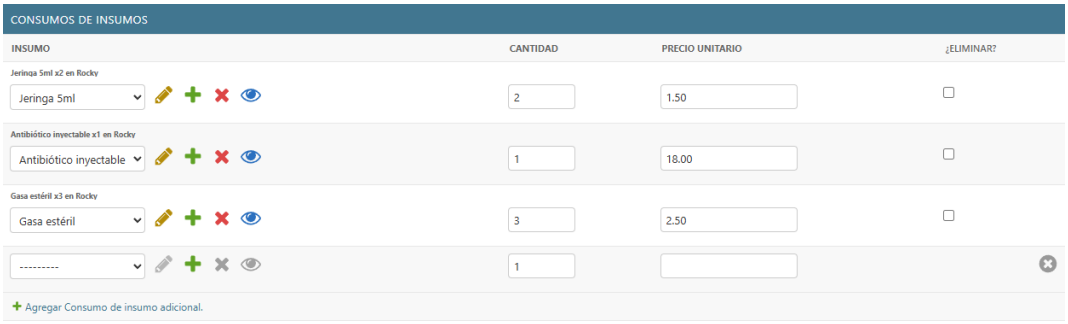
En el Admin se revisó el listado de citas y el de veterinarios. Los consumos se ven dentro de la pantalla de cada cita, en la tabla de insumos usados.

Terminal — carga de datos y los 8 consumos

```
PS ...\django_project\src> python manage.py shell -c "exec(open('cargar_lab07.py', encoding='utf-8').read())"
Citas: 5 | Veterinarios: 3 | Insumos: 4 | Consumos: 8
PS ...\django_project\src> python manage.py shell
>>> for c in ConsumoInsumo.objects.order_by('id'): print(c)
Jeringa 5ml x2 en Rocky
Antibiótico inyectable x1 en Rocky
Gasa estéril x3 en Rocky
Jeringa 5ml x1 en Michi
Vacuna antirrábica x1 en Luna
Jeringa 5ml x1 en Luna
Antibiótico inyectable x2 en Toby
Gasa estéril x2 en Nala
```







Ejercicio 3 — Registrar una operación con transaction.atomic() y F()

Cree una View (con su URL y un Template que herede de base.html) que registre una operación que modifique más de un modelo. Encierre la operación en transaction.atomic() y descuente el campo entero con F(). Si no hay existencia suficiente, la operación debe cancelarse por completo y mostrar el mensaje de error en el Template. Cuando la operación sea exitosa, aplique el patrón Post/Redirect/Get. Pruebe un caso exitoso y uno que falle, y compare la cantidad de registros de cada modelo antes y después del caso que falla.

Respuesta:

Se creó la pantalla Registrar consumo de insumo, con su ruta, su vista y una plantilla que hereda de la plantilla base. Al enviar el formulario, la vista hace tres cosas dentro de un solo bloque transaccional: registra el consumo, marca la cita como Atendida y descuenta el stock del insumo. El descuento se hace dentro de la base de datos con F() y solo se aplica si el stock alcanza; si no alcanza, se lanza un error que cancela todo el bloque y la plantilla muestra el mensaje. Cuando todo sale bien se redirige al detalle de la cita, siguiendo el patrón Post/Redirect/Get.

Caso exitoso: en la cita de Toby se registraron 2 jeringas. La página llevó al detalle de la cita, que ya aparece como Atendida y con la jeringa en su lista. Los consumos pasaron de 8 a 9 y el stock bajó de 100 a 98.

Caso que falla: en la cita de Luna se pidieron 9999 jeringas. Se mostró el mensaje Stock insuficiente de Jeringa 5ml en el mismo formulario. Antes y después había 5 citas, 9 consumos y 98 jeringas, y Luna siguió como Pendiente. El consumo que se había creado primero y el cambio de estado también se deshicieron, así que ningún registro quedó a medias.

vet/views.py — consumo_registrar

```python
def consumo_registrar(request):
    error = None
    if request.method == 'POST':
        form = ConsumoForm(request.POST)
        if form.is_valid():
            datos = form.cleaned_data
            try:
                with transaction.atomic():
                    ConsumoInsumo.objects.create(
                        cita=datos['cita'],
                        insumo=datos['insumo'],
                        cantidad=datos['cantidad'],
                        precio_unitario=datos['precio_unitario'],
                    )
                    Cita.objects.filter(pk=datos['cita'].pk).update(estado='Atendida')
                    filas = Insumo.objects.filter(
                        pk=datos['insumo'].pk,
                        stock__gte=datos['cantidad'],
                    ).update(stock=F('stock') - datos['cantidad'])
                    if filas == 0:
                        raise ValueError('Stock insuficiente de ' + datos['insumo'].nombre)
            except ValueError as e:
                error = str(e)
            else:
                return redirect('vet:cita_detalle', pk=datos['cita'].pk)
    else:
        form = ConsumoForm()
    return render(request, 'vet/consumo_form.html', {'form': form, 'error': error})
```

vet/urls.py

```python
urlpatterns = [
    # /         -> listado de citas
    # /nueva/   -> formulario para registrar
    path('', views.cita_list, name='cita_list'),
    path('nueva/', views.cita_crear, name='cita_crear'),
    # Semana 4: detalle con los datos relacionados (1:1, 1:N y el intermedio).
    path('cita/<int:pk>/', views.cita_detalle, name='cita_detalle'),
    path('consumo/nuevo/', views.consumo_registrar, name='consumo_registrar'),
    path('reporte/', views.reporte, name='reporte'),
]
```

templates/vet/consumo_form.html

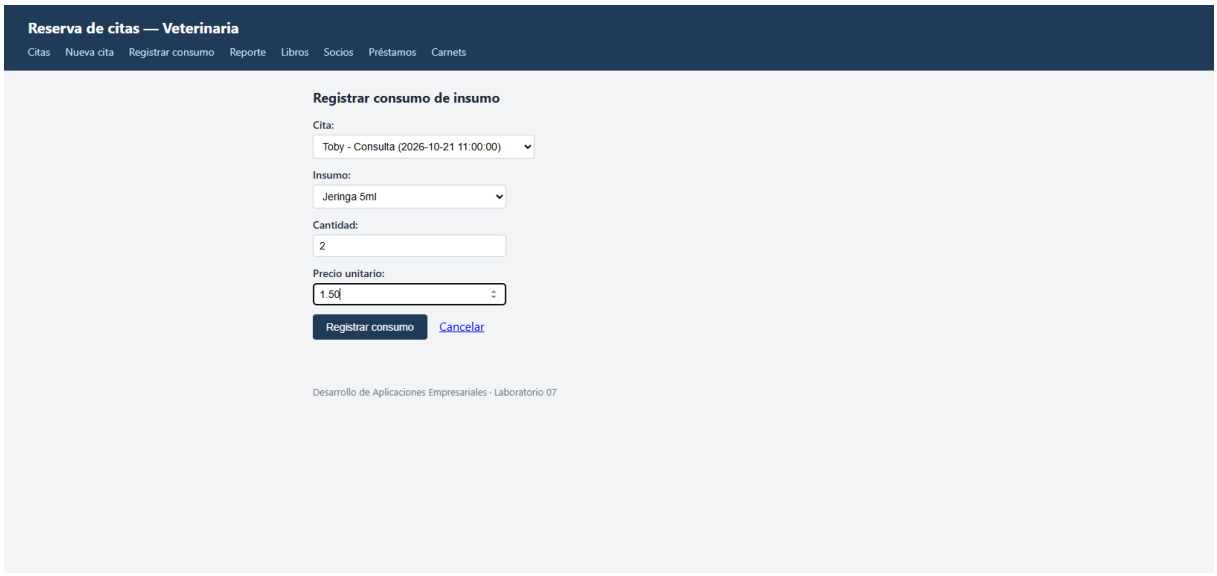
```
{% extends 'base.html' %}

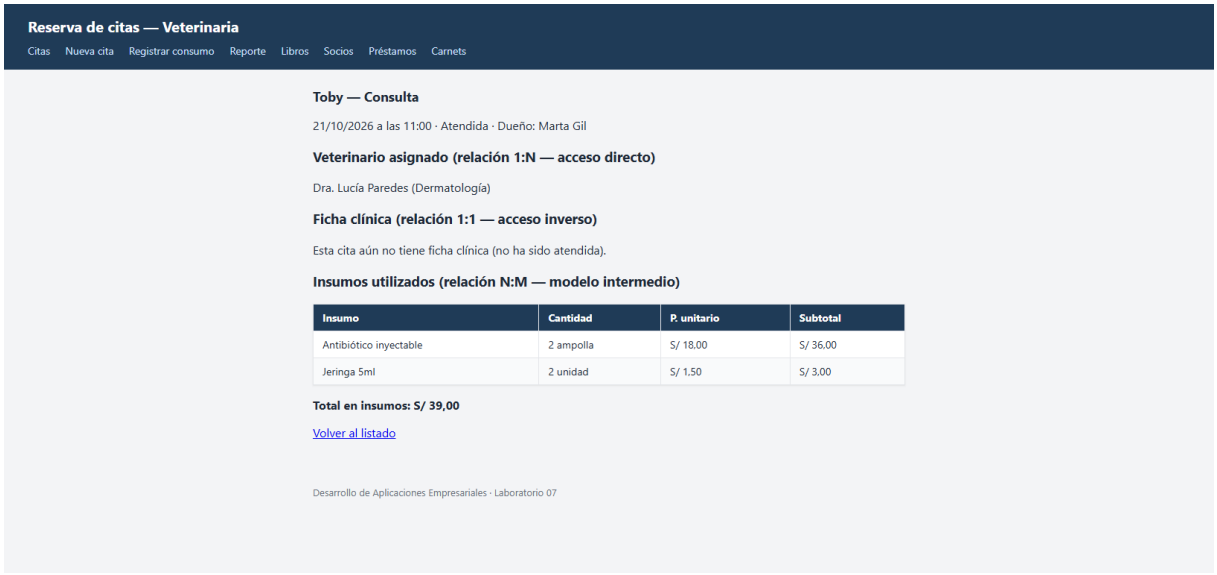
{% block title %}Registrar consumo{% endblock %}

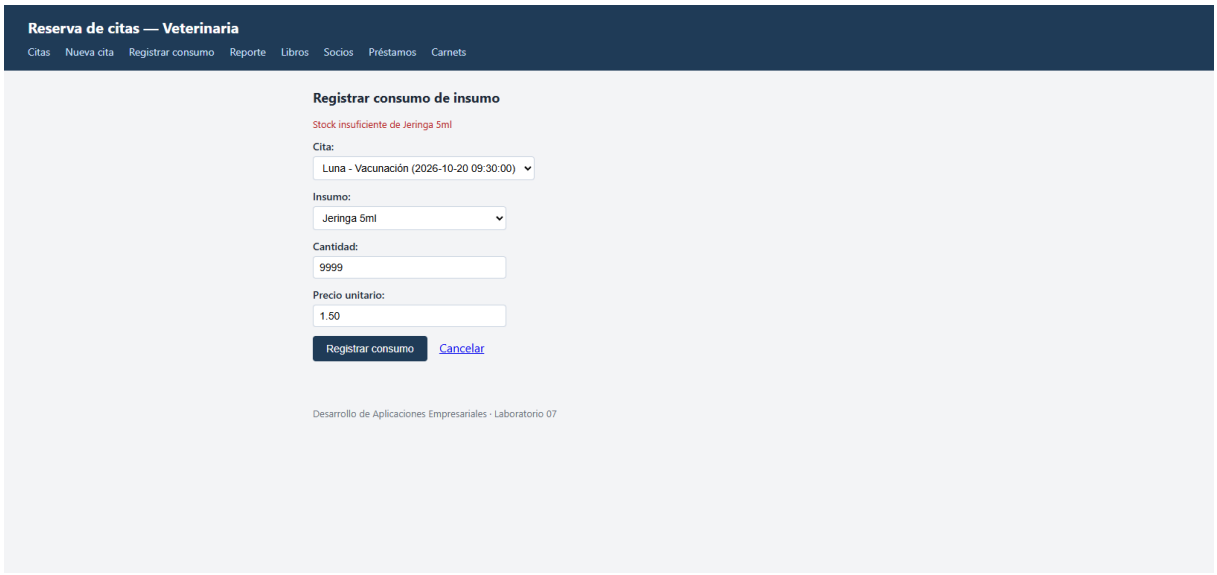
{% block content %}
<h2>Registrar consumo de insumo</h2>

{% if error %}
<p class="errorlist">{{ error }}</p>
{% endif %}

<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Registrar consumo</button>
    <a href="{% url 'vet:cita_list' %}">Cancelar</a>
</form>
{% endblock %}
```







Shell de Django — citas, consumos y stock de la jeringa

```
# Antes del caso exitoso
>>> Cita.objects.count(), ConsumoInsumo.objects.count(), Insumo.objects.get(nombre='Jeringa 5ml').stock
(5, 8, 100)
# Después del caso exitoso (Toby, 2 jeringas)
>>> Cita.objects.count(), ConsumoInsumo.objects.count(), Insumo.objects.get(nombre='Jeringa 5ml').stock
(5, 9, 98)
# Antes del caso que falla
>>> Cita.objects.count(), ConsumoInsumo.objects.count(), Insumo.objects.get(nombre='Jeringa 5ml').stock
(5, 9, 98)
# Después del caso que falla (Luna, 9999 jeringas): nada cambió
>>> Cita.objects.count(), ConsumoInsumo.objects.count(), Insumo.objects.get(nombre='Jeringa 5ml').stock
(5, 9, 98)
```

Ejercicio 4 — Calcular un total con aggregate()

En el shell de Django (python manage.py shell), calcule con aggregate() un total global a partir del modelo intermedio (por ejemplo, la suma de cantidad × monto usando F()). Muestre el diccionario obtenido y explique con sus palabras por qué aggregate() no devuelve un QuerySet.

Respuesta:

En el shell se calculó el total global consumido, sumando cantidad por precio unitario de todos los consumos con F(). El resultado es un diccionario con una sola clave: el total es 110.50 soles.

aggregate() no devuelve un QuerySet porque resume toda la tabla en un único valor. Un QuerySet es una colección de objetos del modelo; un total no es un objeto ni una lista de ellos, así que Django devuelve un diccionario simple. Por eso es una consulta final: ya no se le puede seguir filtrando ni ordenando.

Shell de Django — aggregate()

```
PS ...\django_project\src> python manage.py shell
>>> from django.db.models import Sum, F, DecimalField
>>> ConsumoInsumo.objects.aggregate(
...     total=Sum(F('cantidad') * F('precio_unitario'), output_field=DecimalField()))
{'total': Decimal('110.5')}
```

Ejercicio 5 — Calcular valores por objeto con annotate()

Calcule con annotate() un valor por objeto (por ejemplo, el número de registros relacionados con Count) y un agrupado por estado con values().annotate(). Ordene los resultados de mayor a menor y documente cada consulta con su salida.

Respuesta:

Con annotate() se agregó a cada cita el número de consumos que tiene, ordenado de mayor a menor: Rocky 3, Luna 2, Toby 2, Michi 1 y Nala 1. A diferencia de aggregate(), aquí el resultado sigue siendo un QuerySet de citas y cada cita trae su valor calculado.

Con values() y annotate() se agruparon las citas por estado, también de mayor a menor: Atendida 3, Pendiente 1 y Confirmada 1.

Shell de Django — annotate() y values().annotate()

```
>>> from django.db.models import Count
>>> for c in Cita.objects.annotate(n=Count('consumos')).order_by('-n'):
...     print(c.mascota, c.n)
...
Rocky 3
Luna 2
Toby 2
Michi 1
Nala 1
>>> Cita.objects.values('estado').annotate(total=Count('id')).order_by('-total')
<CitaQuerySet [{'estado': 'Atendida', 'total': 3}, {'estado': 'Pendiente', 'total': 1}, {'estado': 'Confirmada', 'total': 1}]>
```

Ejercicio 6 — Crear la página de reporte

Cree una View de reporte con su URL y un Template reporte.html que herede de base.html y muestre los resultados de los Ejercicios 4 y 5. Aplique al menos un filtro de plantilla (por ejemplo, floatformat) sobre un valor calculado.

Respuesta:

Se creó la ruta del reporte, su vista y la plantilla del reporte, que hereda de la plantilla base. Muestra el total global consumido, el número de consumos por cita, las citas por estado y la cantidad de citas próximas pendientes. El total sale 110,50 soles y se muestra con el filtro que fija dos decimales, de modo que siempre aparece con el formato de un precio. Hay 1 cita próxima pendiente, la de Luna.

vet/views.py — reporte

```python
def reporte(request):
    total = ConsumoInsumo.objects.aggregate(
        total=Sum(F('cantidad') * F('precio_unitario'), output_field=DecimalField())
    )['total']
    por_cita = Cita.objects.annotate(n=Count('consumos')).order_by('-n')
    por_estado = Cita.objects.values('estado').annotate(total=Count('id')).order_by('-total')
    pendientes_proximas = Cita.objects.proximas().con_estado('Pendiente').count()
    return render(request, 'vet/reporte.html', {
        'total': total,
        'por_cita': por_cita,
        'por_estado': por_estado,
        'pendientes_proximas': pendientes_proximas,
    })
```

templates/vet/reporte.html

```
{% extends 'base.html' %}

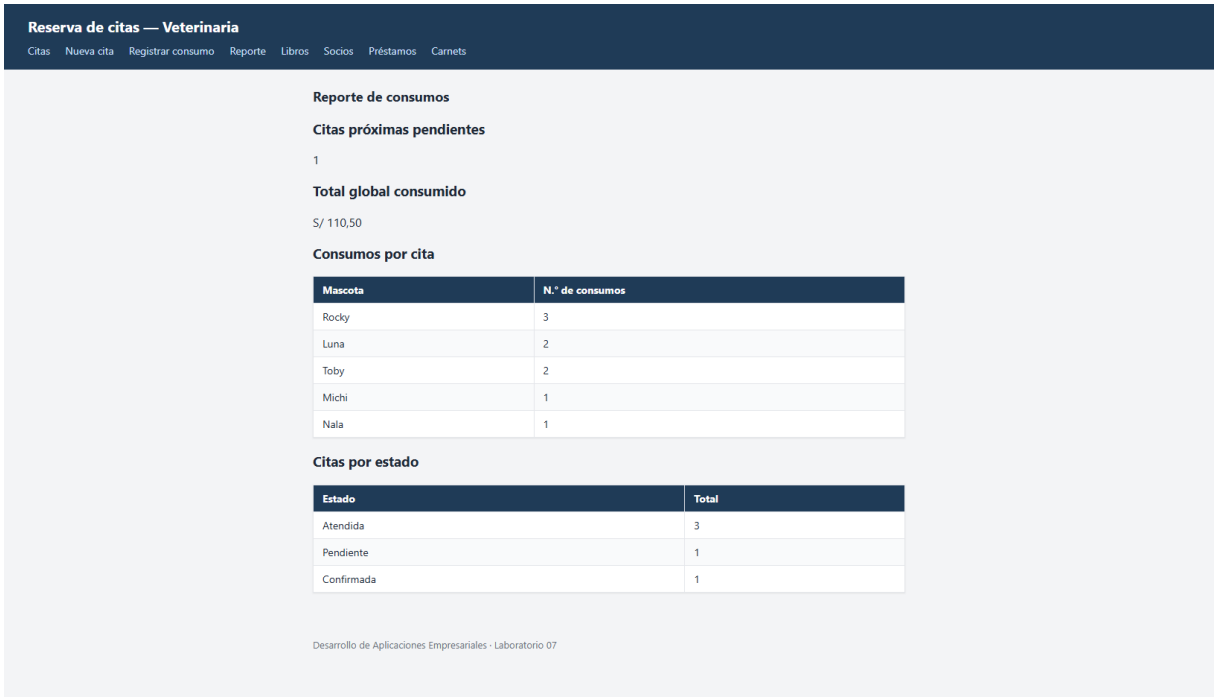
{% block title %}Reporte{% endblock %}

{% block content %}
<h2>Reporte de consumos</h2>
<h3>Citas próximas pendientes</h3>
<p>{{ pendientes_proximas }}</p>

<h3>Total global consumido</h3>
<p>S/ {{ total|floatformat:2 }}</p>

<h3>Consumos por cita</h3>
<table>
    <thead><tr><th>Mascota</th><th>N.° de consumos</th></tr></thead>
    <tbody>
    {% for c in por_cita %}
        <tr><td>{{ c.mascota }}</td><td>{{ c.n }}</td></tr>
    {% endfor %}
    </tbody>
</table>

<h3>Citas por estado</h3>
<table>
    <thead><tr><th>Estado</th><th>Total</th></tr></thead>
    <tbody>
    {% for e in por_estado %}
        <tr><td>{{ e.estado }}</td><td>{{ e.total }}</td></tr>
    {% endfor %}
    </tbody>
</table>
{% endblock %}
```



Ejercicio 7 — Crear un QuerySet personalizado

Cree un QuerySet personalizado para su entidad principal con al menos dos métodos encadenables que representen reglas del negocio (por ejemplo, registros con un estado determinado y registros del mes actual). Asígnelo con as_manager() y reemplace en al menos dos Views las consultas repetidas por sus métodos. Verifique que el CRUD de la Semana 3 sigue funcionando.

Respuesta:

Se creó un QuerySet personalizado para la cita con dos métodos que se pueden encadenar y que representan reglas del negocio: citas con un estado determinado y citas desde hoy en adelante. Se asignó como manager de la cita con as_manager().

Se usa en dos vistas: el listado de citas filtra por el estado que llega en la dirección, y el reporte combina los dos métodos para contar las citas próximas que siguen pendientes. El listado completo, el filtro por estado y el detalle siguen funcionando, así que el CRUD de la Semana 3 no se rompió.

vet/models.py — CitaQuerySet

```python
class CitaQuerySet(models.QuerySet):
    def con_estado(self, estado):
        return self.filter(estado=estado)

    def proximas(self):
        return self.filter(fecha__gte=timezone.localdate())

class Cita(models.Model):
    # Opciones fijas: van en el codigo, no en una tabla aparte.
    SERVICIOS = [
        ('Consulta', 'Consulta'),
        ('Vacunación', 'Vacunación'),
        ('Cirugía', 'Cirugía'),
        ('Baño', 'Baño'),
    ]
    ESTADOS = [
        ('Pendiente', 'Pendiente'),
        ('Confirmada', 'Confirmada'),
        ('Atendida', 'Atendida'),
    ]

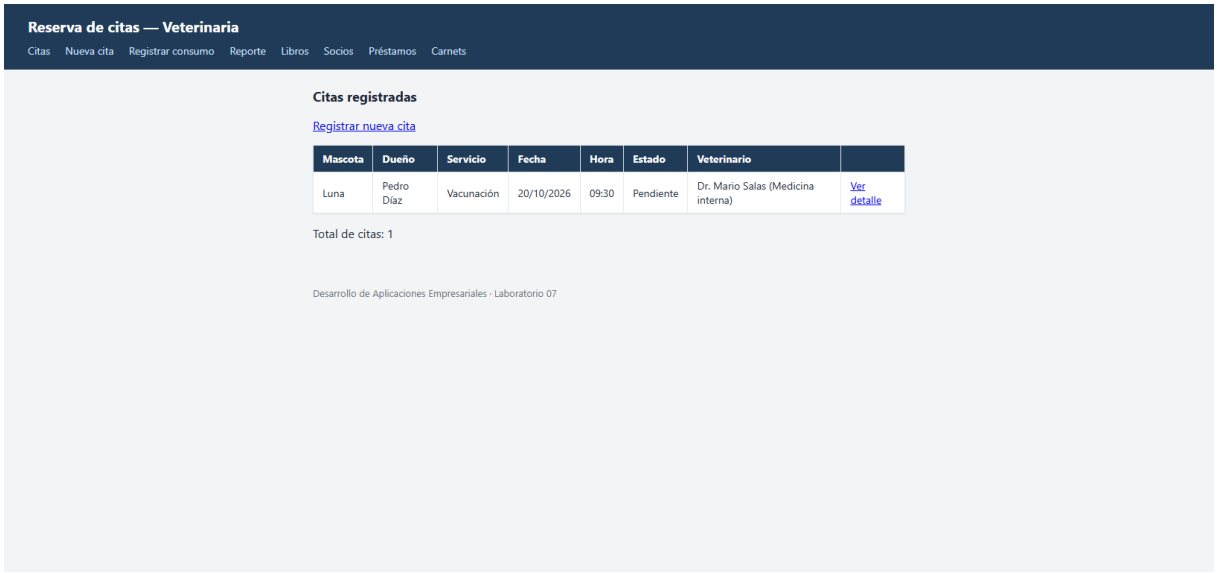
    mascota = models.CharField(max_length=100)
    dueno = models.CharField(max_length=100)
    servicio = models.CharField(max_length=20, choices=SERVICIOS)
    fecha = models.DateField()
    hora = models.TimeField()
    estado = models.CharField(max_length=20, choices=ESTADOS, default='Pendiente')
    objects = CitaQuerySet.as_manager()
```

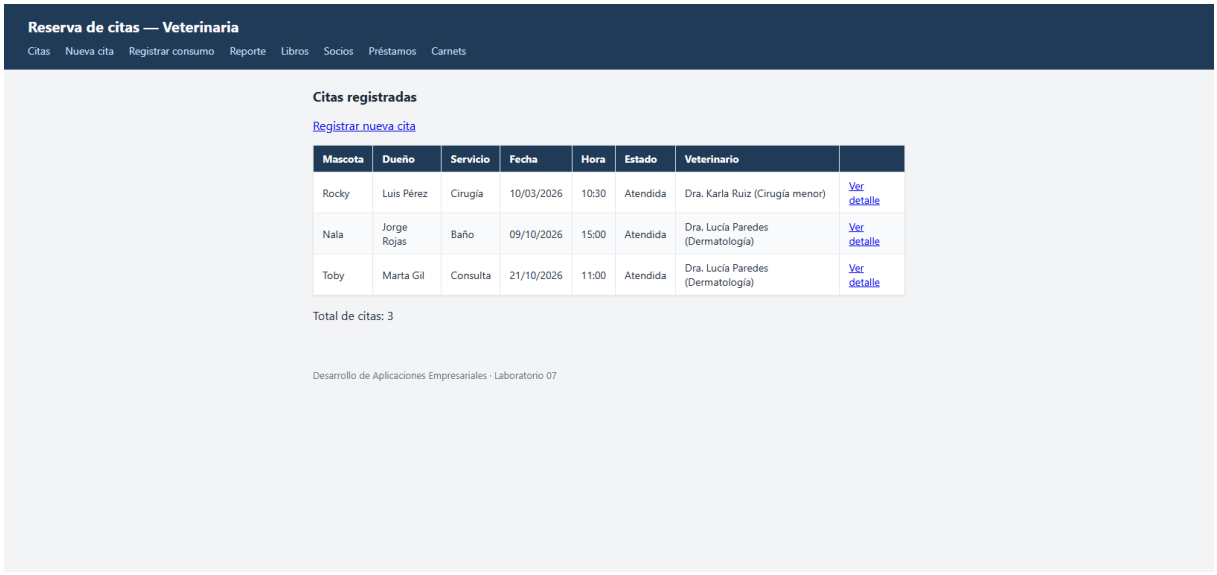
vet/views.py — cita_list usa con_estado()

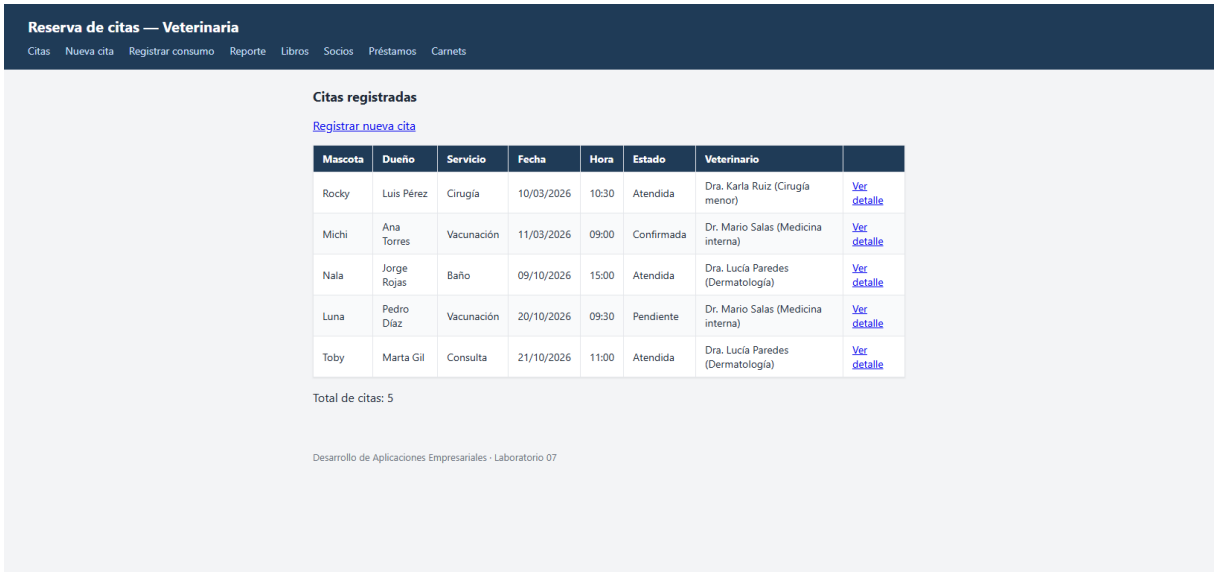
```python
def cita_list(request):
    # Ejercicio 6 - select_related(): resuelve la FK (1:N) y el 1:1 inverso
    # en UNA sola consulta con JOIN, en vez de una consulta extra por fila.
    citas = (Cita.objects
             .select_related('veterinario', 'ficha')
             .order_by('fecha', 'hora'))
    estado = request.GET.get('estado')
    if estado:
        citas = citas.con_estado(estado)
    return render(request, 'vet/cita_list.html', {'citas': citas})
```

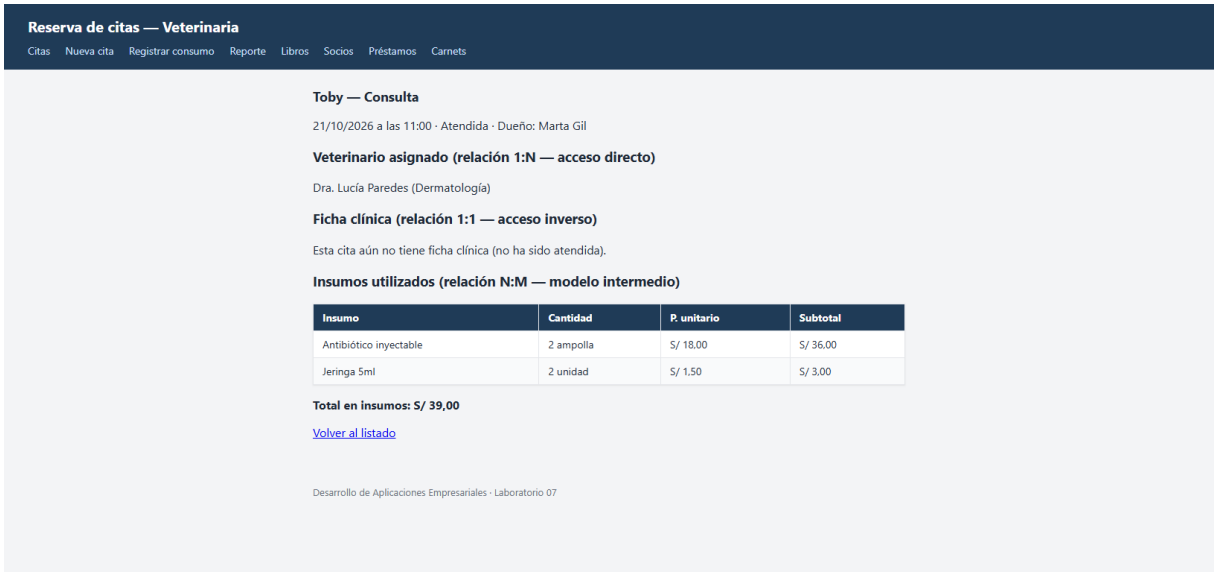
vet/views.py — reporte encadena proximas() y con_estado()

```python
def reporte(request):
    total = ConsumoInsumo.objects.aggregate(
        total=Sum(F('cantidad') * F('precio_unitario'), output_field=DecimalField())
    )['total']
    por_cita = Cita.objects.annotate(n=Count('consumos')).order_by('-n')
    por_estado = Cita.objects.values('estado').annotate(total=Count('id')).order_by('-total')
    pendientes_proximas = Cita.objects.proximas().con_estado('Pendiente').count()
    return render(request, 'vet/reporte.html', {
        'total': total,
        'por_cita': por_cita,
        'por_estado': por_estado,
        'pendientes_proximas': pendientes_proximas,
    })
```









Ejercicio 8 — Medir y optimizar el problema N+1

Con DEBUG=True, use connection.queries (y reset_queries) para contar las consultas que ejecuta un listado que muestra datos de una relación. Optimice el listado con select_related() o prefetch_related() y documente el número de consultas antes y después. Explique con sus palabras cuándo usaría cada método.

Respuesta:

Con DEBUG activo se contaron las consultas con la lista de consultas de la conexión, reiniciándola antes de cada medición. Con las 5 citas y los 9 consumos cargados:

Pantalla	Sin optimizar	Optimizada
Listado con el veterinario de cada cita	6 consultas	1 consulta (select_related)
Citas con sus consumos e insumos	15 consultas	3 consultas (prefetch_related)

Sin optimizar, Django hace una consulta para traer las citas y luego una más por cada cita para llegar a su veterinario (6 = 1 + 5), y en el segundo caso una más por cada cita para sus consumos y otra por cada consumo para su insumo (15 = 1 + 5 + 9). Ese es el problema N+1: el número de consultas crece con la cantidad de filas.

select_related() se usa con relaciones de a uno, como la clave foránea o el uno a uno: une las tablas y trae todo en una sola consulta. prefetch_related() se usa con relaciones de a muchos, como el muchos a muchos o el lado inverso de una clave foránea: hace una consulta por relación y une los resultados en Python, porque unir las tablas repetiría las filas.

En la aplicación, el listado usa select_related() para el veterinario y la ficha, y el detalle de la cita usa prefetch_related() para sus consumos y el insumo de cada consumo.

medir_n1.py — conteo de consultas

```python
from django.db import connection, reset_queries
from vet.models import Cita

reset_queries()
for c in Cita.objects.all():
    nombre = c.veterinario.nombre if c.veterinario else '-'
print('Veterinario SIN optimizar:', len(connection.queries))

reset_queries()
for c in Cita.objects.select_related('veterinario'):
    nombre = c.veterinario.nombre if c.veterinario else '-'
print('Veterinario CON select_related:', len(connection.queries))

reset_queries()
for c in Cita.objects.all():
    for k in c.consumos.all():
        n = k.insumo.nombre
print('Consumos SIN optimizar:', len(connection.queries))

reset_queries()
for c in Cita.objects.prefetch_related('consumos__insumo'):
    for k in c.consumos.all():
        n = k.insumo.nombre
print('Consumos CON prefetch_related:', len(connection.queries))
```

Terminal — consultas antes y después de optimizar

```
PS ...\django_project\src> python manage.py shell -c "exec(open('medir_n1.py', encoding='utf-8').read())"
Veterinario SIN optimizar: 6
Veterinario CON select_related: 1
Consumos SIN optimizar: 15
Consumos CON prefetch_related: 3
```

vet/views.py — listado y detalle optimizados

```python
def cita_list(request):
    # Ejercicio 6 - select_related(): resuelve la FK (1:N) y el 1:1 inverso
    # en UNA sola consulta con JOIN, en vez de una consulta extra por fila.
    citas = (Cita.objects
             .select_related('veterinario', 'ficha')
             .order_by('fecha', 'hora'))
    estado = request.GET.get('estado')
    if estado:
        citas = citas.con_estado(estado)
    return render(request, 'vet/cita_list.html', {'citas': citas})


def cita_detalle(request, pk):
    # Ejercicio 6 - prefetch_related(): recorre el modelo intermedio.
    # Django lanza una consulta extra para los ConsumoInsumo y otra para los
    # Insumo, y une los resultados en Python.
    cita = get_object_or_404(
        Cita.objects
            .select_related('veterinario', 'ficha')
            .prefetch_related('consumos__insumo'),
        pk=pk,
    )
    total = sum(c.subtotal for c in cita.consumos.all())
    return render(request, 'vet/cita_detalle.html',
                  {'cita': cita, 'total': total})
```

PARTE 2 — Consultas avanzadas sobre la investigación propia (registrada en el Admin en la Semana 5)

Esta parte se desarrolla sobre la app biblioteca y está a cargo de Lucas Inga.

A partir de aquí trabajará sobre la base de datos de su investigación propia: las cinco entidades originales, la entidad de la relación OneToOneField y el modelo intermedio de la relación ManyToManyField. El objetivo es aplicar las mismas técnicas sobre ese modelo de datos, cumpliendo obligatoriamente el checklist de la siguiente sección antes de continuar.

Ejercicio 9 — Identificar dónde aplica cada técnica en tu investigación

Complete la tabla de equivalencias con su investigación: indique qué campo entero descontará, qué campo de estado usará, qué atributos del modelo intermedio servirán para calcular totales y qué operación modificará más de un modelo a la vez. Si algún campo no existe, agréguelo con makemigrations y migrate.

Respuesta:

Se trabajó con la app biblioteca del repositorio. No hizo falta agregar campos nuevos: la migración 0003_libro_copias_prestamo_multa ya creó copias y multa.

Elemento pedido	En la investigación (biblioteca)
Campo entero que se descontará	Libro.copias
Campo de estado	Prestamo.estado (Activo, Devuelto, Atrasado, Extraviado)

Atributos del modelo intermedio para totales	Prestamo.multa (suma) y el conteo de préstamos por libro y por estado
Operación que modifica más de un modelo	Registrar un préstamo: crea el Prestamo y descuenta Libro.copias

Checklist antes de continuar — Parte 2

Criterio	Verificación
1. La operación de tu investigación se ejecuta dentro de transaction.atomic() y descuenta un campo con F().	Código de la View + captura.
2. Se comprobó que un error provoca el rollback completo (ningún registro quedó a medias).	Capturas del conteo de registros antes y después.
3. Se calculó al menos un total con aggregate() sobre el modelo intermedio.	Código + salida en el shell.
4. Se calculó al menos un valor por objeto con annotate() y un agrupado con values().annotate().	Código + captura de la salida.
5. Existe un Template de reporte que hereda de base.html y muestra al menos dos reportes.	Captura del Template renderizado.
6. Existe un QuerySet personalizado con al menos dos métodos encadenables, asignado con as_manager().	Código de models.py.
7. Al menos dos Views usan los métodos del QuerySet personalizado y el CRUD sigue funcionando.	Prueba funcional antes/después.
8. Se midió el número de consultas de una pantalla antes y después de optimizar con select_related o prefetch_related.	Capturas de len(connection.queries).
9. Se explica por escrito qué resuelve cada técnica en esta investigación.	Párrafo de justificación.

Ejercicio 10 — Transacción en la investigación propia

Implemente en su investigación la operación transaccional identificada en el Ejercicio 9 (con View, URL y un Template que herede de base.html), usando transaction.atomic() y F(). Pruebe un caso exitoso y uno que provoque rollback, y documente ambos con capturas.

Respuesta:

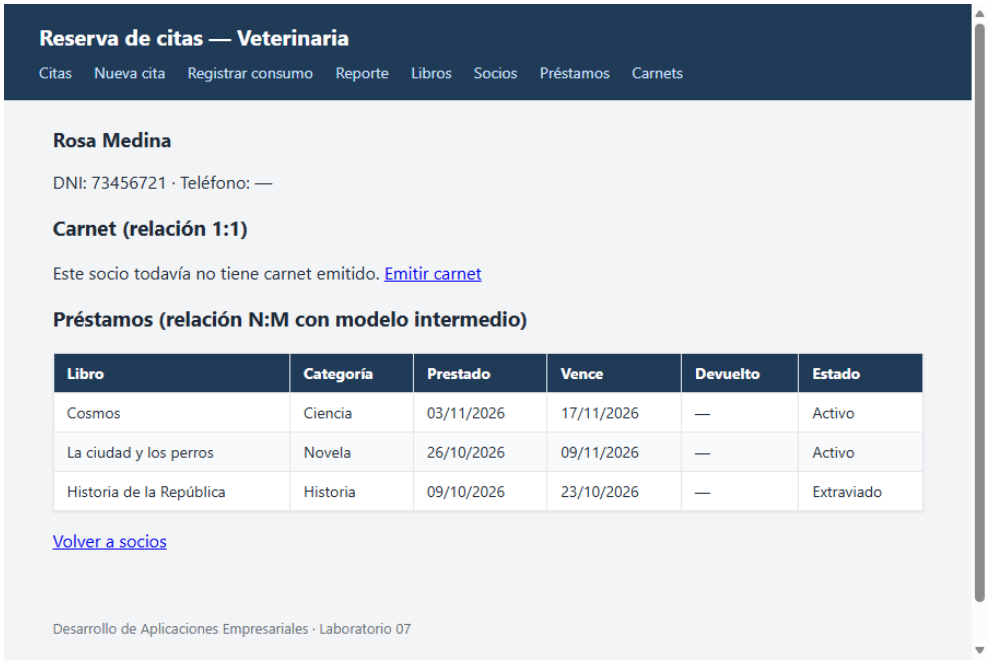
Se creó la pantalla Registrar préstamo (ruta /biblioteca/prestamos/registrar/, vista prestamo_registrar y plantilla que hereda de base.html). Dentro de transaction.atomic() se crea el Prestamo y se descuenta una copia con F(); si el UPDATE no afecta filas (no hay copias), se lanza un error y se deshace todo.

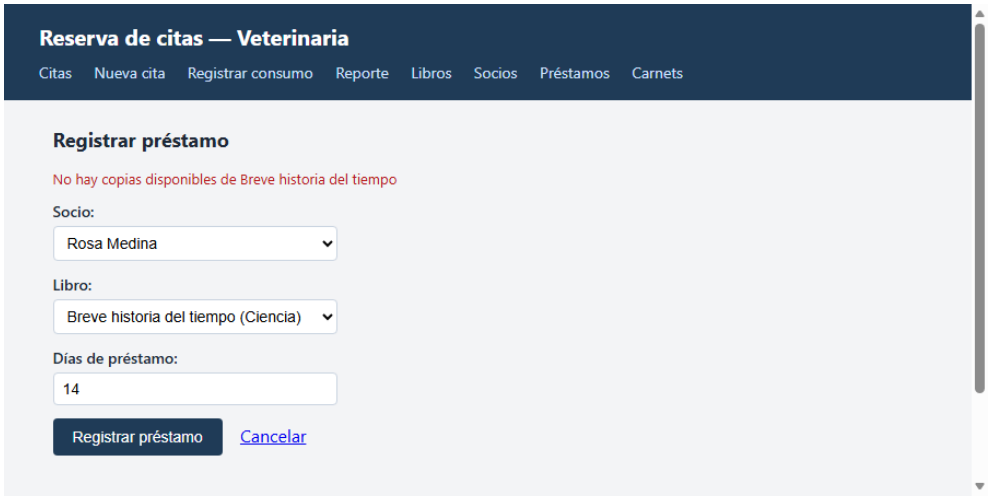
Caso exitoso: Rosa Medina pide Cosmos; los préstamos pasan de 9 a 10 y las copias de Cosmos de 6 a 5.
Caso con rollback: Breve historia del tiempo sin copias; se muestra el mensaje de error y los préstamos siguen en 10, es decir, ningún registro quedó a medias.

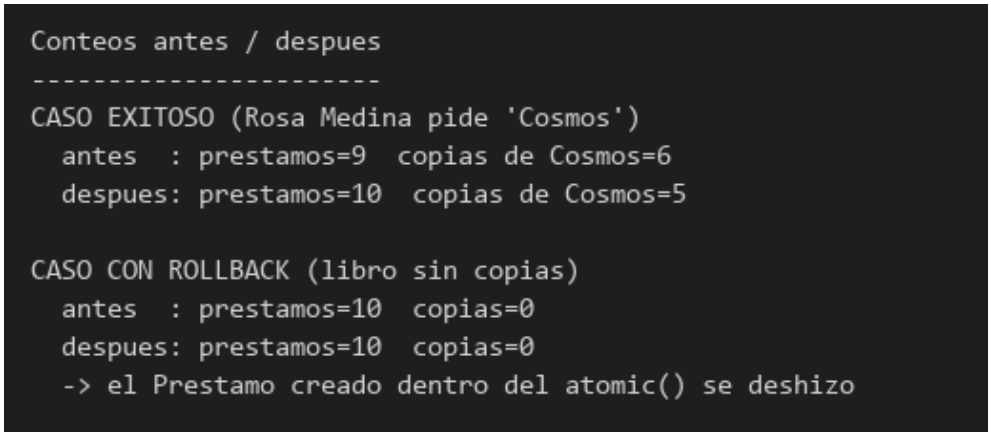
```
biblioteca/views.py  (líneas 271-298)
-------------------------------------
 271  def prestamo_registrar(request):
 272      error = None
 273      if request.method == 'POST':
 274          form = PrestarForm(request.POST)
 275          if form.is_valid():
 276              datos = form.cleaned_data
 277              hoy = date.today()
 278              try:
 279                  with transaction.atomic():
 280                      Prestamo.objects.create(
 281                          socio=datos['socio'],
 282                          libro=datos['libro'],
 283                          fecha_prestamo=hoy,
 284                          fecha_devolucion_prevista=hoy + timedelta(days=datos['dias']),
 285                          estado='Activo',
 286                      )
 287                      filas = Libro.objects.filter(
 288                          pk=datos['libro'].pk,
 289                          copias__gte=1,
 290                      ).update(copias=F('copias') - 1)
 291                      if filas == 0:
 292                          raise ValueError('No hay copias disponibles de ' + datos['libro'].titulo)
 293              except ValueError as e:
 294                  error = str(e)
 295              else:
 296                  return redirect('biblioteca:socio_detail', pk=datos['socio'].pk)
 297      else:
 298          form = PrestarForm()
```

```
biblioteca/urls.py  (líneas 36-39)
----------------------------------
  36      path('prestamos/', views.PrestamoListView.as_view(), name='prestamo_list'),
  37      path('prestamos/nuevo/', views.PrestamoCreateView.as_view(), name='prestamo_crear'),
  38      path('prestamos/registrar/', views.prestamo_registrar, name='prestamo_registrar'),
  39      path('reporte/', views.reporte, name='reporte'),
```

```
templates/biblioteca/prestamo_registrar.html  (líneas 1-17)
-----------------------------------------------------------
   1  {% extends 'base.html' %}
   2
   3  {% block title %}Registrar préstamo{% endblock %}
   4
   5  {% block content %}
   6  <h2>Registrar préstamo</h2>
   7
   8  {% if error %}
   9  <p class="errorlist">{{ error }}</p>
  10  {% endif %}
  11
  12  <form method="post">
  13      {% csrf_token %}
  14      {{ form.as_p }}
  15      <button type="submit">Registrar préstamo</button>
  16      <a href="{% url 'biblioteca:prestamo_list' %}">Cancelar</a>
  17  </form>
```







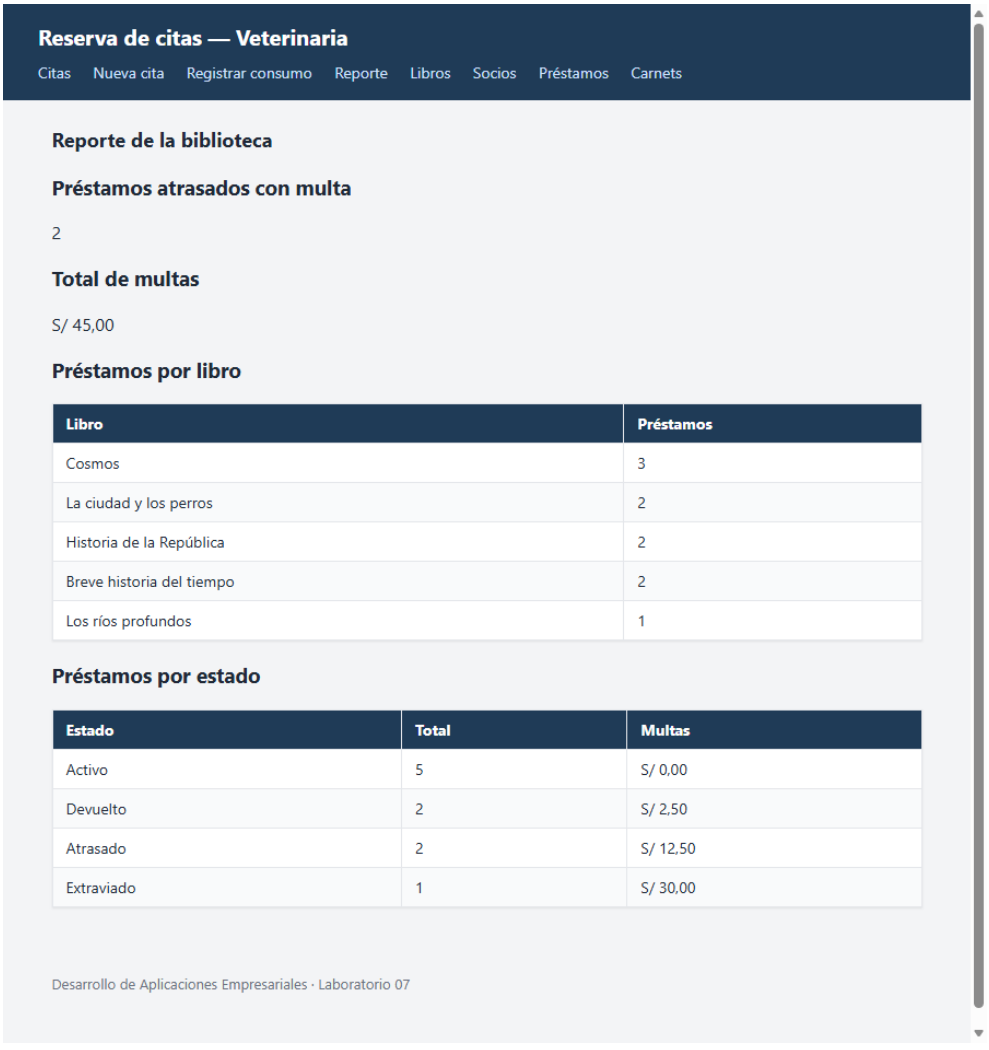
Ejercicio 11 — Reportes de la investigación propia

Cree al menos dos reportes (uno con aggregate() y otro con annotate() o values().annotate()) y muéstrelos en un Template que herede de base.html.

Respuesta:

La vista reporte combina aggregate() (total de multas con Sum), annotate() (préstamos por libro con Count) y values().annotate() (total y multas por estado). Se muestran en un Template que hereda de base.html con dos tablas.

```
biblioteca/views.py  (líneas 301-312)
-------------------------------------
 301  def reporte(request):
 302      total_multas = Prestamo.objects.aggregate(total=Sum('multa'))['total']
 303      por_libro = Libro.objects.annotate(n=Count('prestamos')).order_by('-n')
 304      por_estado = (Prestamo.objects.values('estado')
 305                    .annotate(total=Count('id'), multas=Sum('multa'))
 306                    .order_by('-total'))
 307      atrasados_con_multa = Prestamo.objects.con_estado('Atrasado').con_multa().count()
 308      return render(request, 'biblioteca/reporte.html', {
 309          'total_multas': total_multas,
 310          'por_libro': por_libro,
 311          'por_estado': por_estado,
 312          'atrasados_con_multa': atrasados_con_multa,
```



Ejercicio 12 — QuerySet personalizado en la investigación propia

Cree un QuerySet personalizado para una de sus entidades con al menos dos métodos encadenables y úselo en al menos dos Views.

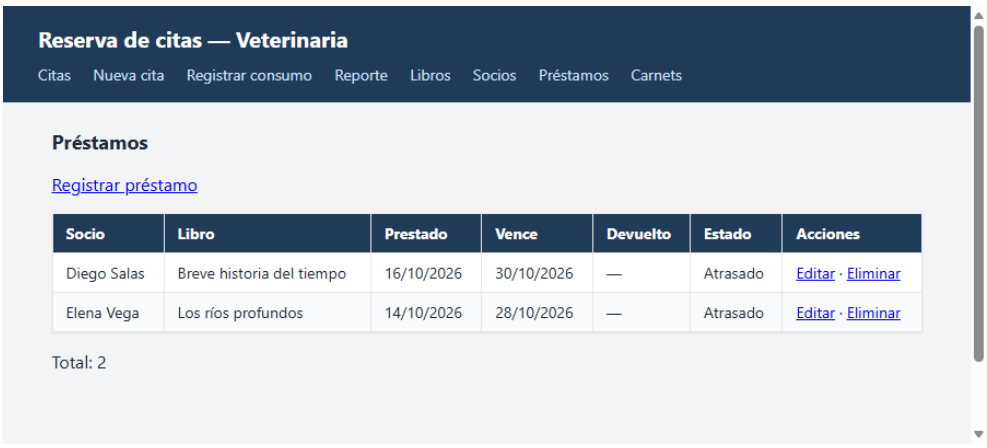
Respuesta:

PrestamoQuerySet tiene dos métodos encadenables, con_estado() y con_multa(), y se asigna con objects = PrestamoQuerySet.as_manager(). Lo usan PrestamoListView (filtro ?estado=) y la vista reporte (con_estado("Atrasado").con_multa().count()). El CRUD de préstamos sigue funcionando.

```
biblioteca/models.py  (líneas 115-121)
--------------------------------------
 115  class PrestamoQuerySet(models.QuerySet):
 116      def con_estado(self, estado):
 117          return self.filter(estado=estado)
 118
 119      def con_multa(self):
 120          return self.filter(multa__gt=0)
 121
```

```
biblioteca/models.py  (líneas 148-148)
--------------------------------------
 148      objects = PrestamoQuerySet.as_manager()
```

```
biblioteca/views.py  (líneas 202-213)
-------------------------------------
 202  class PrestamoListView(ListView):
 203      model = Prestamo
 204      template_name = 'biblioteca/prestamo_list.html'
 205      context_object_name = 'objetos'
 206      queryset = Prestamo.objects.select_related('socio', 'libro')
 207      def get_queryset(self):
 208          qs = super().get_queryset()
 209          estado = self.request.GET.get('estado')
 210          if estado:
 211              qs = qs.con_estado(estado)
 212          return qs
 213
```



Ejercicio 13 — Optimizar una pantalla de la investigación propia

Identifique la pantalla de su investigación que ejecuta más consultas, mida cuántas ejecuta,

optimícela con select_related() o prefetch_related() y documente el número de consultas antes y después.

Respuesta:

Las pantallas con más consultas eran el listado de préstamos (socio y libro por cada fila) y el de libros (categoría y préstamos con su socio). Se optimizaron con select_related('socio', 'libro') y con select_related('categoria').prefetch_related('prestamos__socio'). Medido con len(connection.queries) y 10 préstamos: préstamos 21 → 1 consultas; libros 21 → 3 consultas.

```
python manage.py shell < medir_n1_bib.py
----------------------------------------
Prestamos SIN optimizar: 21
Prestamos CON select_related: 1
Libros SIN optimizar: 21
Libros CON select_related + prefetch_related: 3
```

Justificación: transaction.atomic() evita préstamos a medias; F() descuenta la copia dentro de la base de datos; aggregate() y annotate() arman el reporte sin SQL; el QuerySet personalizado centraliza las reglas de préstamos; y select_related/prefetch_related eliminan el problema N+1.

Ejercicio 14 — Publicar en GitHub

Actualice requirements.txt y README.md describiendo qué operación transaccional, qué reportes y qué QuerySets personalizados se implementaron, y sobre qué entidades. Haga commit y push al repositorio. Incluya en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

Respuesta:

El código de la app biblioteca y la descripción del Laboratorio 7 (transacción, reporte, QuerySet personalizado y optimización) están en el README.md del repositorio, junto con requirements.txt. URL del repositorio: https://github.com/RXDG2908/DesAplEmp-S02

Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente el código de models.py y views.py, el reporte renderizado y el número de consultas antes y después de optimizar).
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

Repositorio: https://github.com/RXDG2908/DesAplEmp-S02

Conclusiones:

Registrar una operación que toca varios modelos dentro de una transacción evita datos a medias: si el stock no alcanza, nada se guarda.

Descontar con F() hace la resta dentro de la base de datos y evita perder descuentos cuando dos registros ocurren a la vez.
aggregate() resume toda la tabla en un valor y annotate() calcula un valor por cada objeto o grupo; con ambos se armó el reporte sin escribir SQL.
Un QuerySet personalizado concentra las reglas del negocio en un solo lugar y permite encadenarlas, en vez de repetir los mismos filtros en cada vista.
Medir el número de consultas deja ver el problema N+1 con números: de 15 consultas se bajó a 3 y de 6 a 1.
Las vistas y plantillas de semanas anteriores siguen funcionando: el listado, el detalle y el formulario de citas no se rompieron.